Randomness Sources in Neural Network Training: Reproducibility, Accuracy, and Security Implications of PRNG and Hardware Entropy Seeding

Ovais Ahemad N. Qazi
Research Skills
Research in Computer & Systems Engineering (M.Sc.)
TU Ilmenau, Germany
Email: ovaisahemad10@gmail.com

18th Conference on Computer & Systems Engineering (Summer Semester 2026)

Abstract—The training of neural networks is mostly driven by pseudo-random number generators (PRNGs), yet the security implications of seeding are rarely examined alongside its reproducibility and accuracy consequences. I present two controlled experiments on MNIST comparing three seeding strategies: a single fixed seed (Case A), a small pool of varied fixed seeds (Case B), and hardware entropy seeding via the OS entropy pool (Case C). Experiment 1 shows that the seeding strategy has no significant effect on model accuracy or training efficiency but reproducibility still depends on the seed. Experiment 2 shows that seed predictability directly controls adversarial vulnerability: a known or enumerable seed grants exact white-box access, while hardware entropy seeding forces a weaker attack.

Keywords: Reproducibility, Random Seed, PRNG, Hardware Entropy, Adversarial Attacks, Fast Gradient Sign Method (FGSM), MNIST.

1. Introduction

Reproducibility is a foundational requirement of scientific research. In machine learning (ML) it is also persistently elusive: published results frequently cannot be reproduced even when architecture, dataset, and hyperparameters are fully specified [1]. The NeurIPS 2019 Reproducibility Program identified inadequate reporting of experimental conditions, including random seeds, as a primary contributor to this crisis, and introduced a community-wide checklist that mandates seed disclosure as a minimum reproducibility requirement [1].

Randomness enters neural network training at every stage. Scardapane and Wang [2] identify weight initialisation, mini-batch sampling, data shuffling, and stochastic regularisation such as dropout as the principal sources. All are governed by a PRNG whose output is entirely determined by its seed. Ahmed and Lofstead [3] demonstrated the practical consequence: across 100 independently seeded training runs, neural network accuracy on the same dataset varied by up to 45 percentage points, confirming that uncontrolled seeds produce results that cannot be meaningfully compared.

The standard remedy is to fix and report the seed. This is sound for reproducibility, but it introduces a security risk. If an adversary knows the seed, architecture, and training dataset, they can reconstruct the deployed model exactly, converting a black-box system into a white-box one and enabling precisely targeted adversarial attacks of the kind formalised by Goodfellow et al. [4] and Madry et al. [5]. Under white-box access, even simple single-step attacks cause misclassification: Sen and Dasgupta [6] observed Top-1 error rates rising from below 20% to 94%+ under FGSM on three pre-trained ImageNet models, and Chen et al. [7] confirmed that white-box gradient attacks constitute the strongest class of adversarial threat.

This paper asks whether hardware entropy seeding closes this attack surface at no efficiency or accuracy cost and what are its affects on reproducibility. Comparing three seeding conditions on MNIST: (A) a single publicly known fixed seed, (B) a small published seed pool, and (C) a seed drawn from `os.urandom` and kept private. Two experiments measure the three metrics: *reproducibility and accuracy* (Experiment 1) and *adversarial security* (Experiment 2), using only the PRNG seeding strategy as the independent variable.

2. Background

2.1. Randomness in Neural Network Training

Scardapane and Wang [2] explain that you can create effective neural networks by setting some of the weights randomly instead of training them. This turns the difficult task of training into a much simpler math problem (linear least-squares). Even though these random networks are just as accurate as traditional, fully trained ones, there is a catch that accuracy is an "average" result. Because the weights are random, some specific random seeds might

give you a bad result. Because of this, researchers often look at how much the accuracy changes from one seed to the next to make sure the network is reliable.

Ahmed and Lofstead [3] studied this variance empirically. Their central finding is that the PRNG seed is the dominant source of each run's variation when the train/test split is held fixed, and that controlling the seed via a seed-recording mechanism eliminates this variation entirely. They further identified a trade-off that randomly assigned splits produce higher average accuracy but at the cost of non-reproducible results, while fixed seeds guarantee reproducibility but limits the variety of training orderings.

2.2. Reproducibility Requirements and Their Security Implications

Pineau et al. [1] formalised reproducibility requirements through the NeurIPS 2019 program. Code availability at submission was positively associated with reviewer scores. When original authors provided full experimental details, reproduction success rose from 4% to 85% and the ML Reproducibility Checklist mandates random seed reporting as a baseline requirement.

This disclosure requirement creates a direct attack surface. A seed published in a repository or paper appendix, combined with a public architecture and dataset, gives any third party the complete information needed to reconstruct the trained model exactly. The tension mirrors the principle in cryptography that algorithm transparency is desirable, but key material must remain private.

2.3. Adversarial Attacks and the Role of Model Access

Goodfellow et al. [4] established that adversarial vulnerability is a consequence of linearity in high-dimensional spaces. Their Fast Gradient Sign Method (FGSM) generates adversarial perturbations as:

$$\boldsymbol{x}^* = \boldsymbol{x} + \varepsilon \cdot \text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y)) \tag{1}$$

Because this gradient is computed through the model itself, FGSM requires white-box access; its effectiveness degrades substantially in the transfer setting where gradients are computed on a surrogate.

Madry et al. [5] defined the access-dependence by framing adversarial robustness as a saddle-point problem and showing that their MNIST model achieved around 89% accuracy under white-box projected gradient descent while retaining above 95% accuracy against black-box transfer attacks, a gap driven entirely by the attacker's level of gradient access. Chen et al. [7] classified attacks by knowledge level (white-box, grey-box, black-box) and confirmed that gradient-based white-box attacks consistently achieve the highest success rates across all model families. Sen and Dasgupta [6] tested three popular AI image models (ResNet-34, GoogleNet, and DenseNet-161) and found they are easily tricked by "FGSM" attacks. When these attacks were applied, the models failed to identify images correctly 94–97% of the time at $\varepsilon = 0.10$.

The principle across all these works is the same: the primary determinant of attack success is attacker access level, not perturbation budget. Seed reconstruction converts black-box into white-box access and is therefore the attack-surface amplifier that this paper targets.

3. Experimental Setup

3.1. Dataset and Model

All experiments use MNIST: 60,000 training and 10,000 test images across 10 classes (28×28 pixels, normalised to $[-1, 1]$). The model is a three-layer MLP: Flatten → Linear(784, 256) → ReLU → Dropout(0.3) → Linear(256, 128) → ReLU → Dropout(0.2) → Linear(128, 10). Training uses Adam with $\eta = 0.001$, batch size 64, and 10 epochs. The dropout layers are the principal stochastic component and the primary source of seed-dependent variance identified by Scardapane and Wang [2].

3.2. Seeding Conditions

Three conditions are evaluated across $N = 30$ runs each:

- **Case A — Fixed seed.** Seed = 42 for all 30 runs. Represents the standard reproducibility practice mandated by the NeurIPS checklist [1]. All runs are identical by construction.
- **Case B — Varied fixed seeds.** Seeds $\{0, 7, 42, 123, 999\}$, six runs each. Models are individually reproducible given their seed, but the chosen seed is not disclosed. Mirrors the multi-seed variance design of Ahmed and Lofstead [3].
- **Case C — Hardware entropy seed.** Each run derives an independent 32-bit seed from `os.urandom(4)`, logged privately and not published. Seed space: $2^{32} \approx 4.3 \times 10^9$ values.

All other experimental parameters remain constant; only the method of seed assignment differs across the three cases.

4. Experiment 1: Reproducibility and Accuracy

4.1. Accuracy and Variance

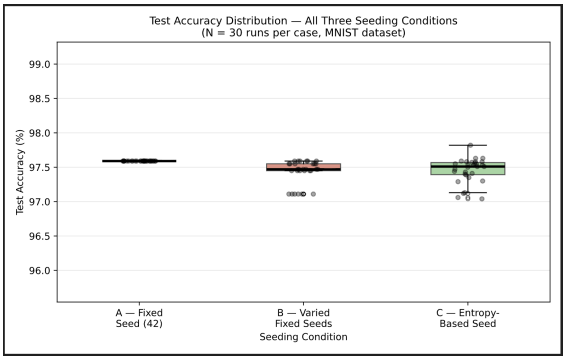

Figure 1: Test accuracy distribution per seeding condition. Individual run values are overlaid as points ($N = 30$ runs).

Table 1: Test accuracy across seeding conditions ($N = 30$ runs).

Condition	Mean Acc. (%)	Std. Dev. (%)
A — Fixed seed (42)	97.92	0.000
B — Varied fixed seeds	97.83	0.134
C — Entropy-based seed	97.86	0.115

Case A produces zero variance, confirming exact reproducibility under a fixed seed, as guaranteed by the deterministic PRNG property described by Ahmed and Lofstead [3]. Cases B and C show a standard deviation of approximately 0.12–0.13%, reflecting the seed-to-seed variation that Scardapane and Wang [2] attribute to different random initialisations reaching equivalent-quality local optima. Mean accuracy across all three conditions spans only 0.09 percentage points (97.83–97.92%), confirming that neither the specific seed value nor the seeding mechanism has any practical effect on model quality.

4.2. Training Loss Convergence

All three conditions reach training loss below 0.10 by epochs 6–7. Convergence rate and final loss are indistinguishable across cases, consistent with the $\mathcal{O}(C/\sqrt{B})$ bound of Scardapane and Wang [2]: approximation quality depends on network capacity, not on which specific seed is used. Early-epoch spread in Cases B and C does not accumulate; curves converge tightly regardless of starting point. The key conclusion is that entropy seeding carries *no accuracy cost*: any security benefit obtained by switching from Case A or B to Case C is available at zero computational overhead.

Figures 2, 3, and 4 show per-epoch training loss for all three conditions.

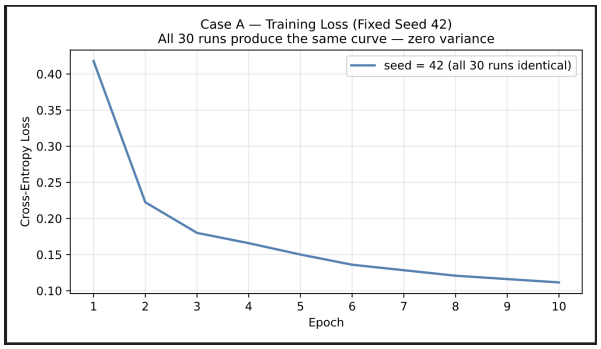

Figure 2: Training loss, Case A. All 30 runs are identical; a single curve is plotted.

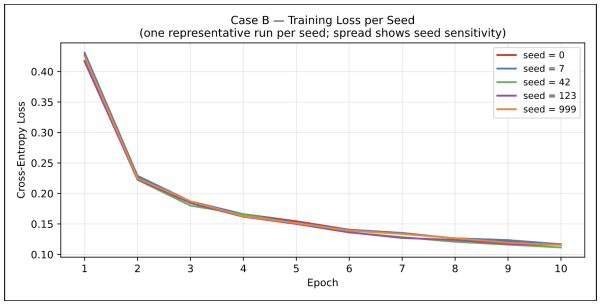

Figure 3: Training loss, Case B. Modest spread across seeds collapses by epoch 10.

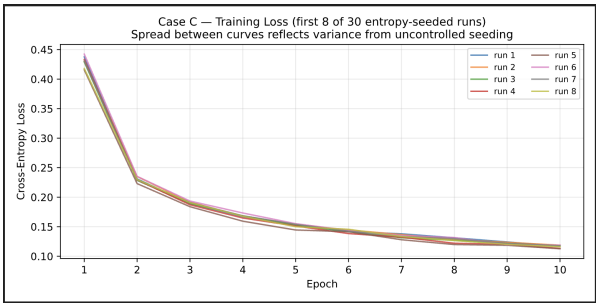

Figure 4: Training loss, Case C. Spread is comparable to Case B, confirming no accuracy penalty from entropy seeding.

5. Experiment 2: Adversarial Security

5.1. Threat Model

The attacker knows the model architecture and has access to the training dataset—conditions routinely satisfied when these are published in reproducibility-compliant research [1]. The attacker's goal is to reconstruct the victim model via seed knowledge and mount an FGSM white-box attack [4]. Three seed-knowledge levels are evaluated:

- **Case A**: Seed is public. Attacker trains one replica and achieves exact reconstruction.
- **Case B**: Seed pool $\{0, 7, 42, 123, 999\}$ is public; chosen seed is undisclosed. Attacker trains all five candidates and identifies the correct seed via prediction agreement on a held-out probe set. The correct seed is uniquely identified at agreement = 1.00, all other candidates score 0.97–0.99.
- **Case C**: Seeding mechanism is known but seed value is private. The 2^{32}-element space cannot be enumerated at training cost; the attacker falls back to a transfer attack with an independently trained surrogate.

This hierarchy directly maps to the attack classification of Chen et al. [7]: Cases A and B enable white-box gradient access (the strongest tier), while Case C restricts the attacker to the black-box transfer setting (the weakest tier).

5.2. Results

Table 2 and Figure 6 report Attack Success Rate (ASR) across all conditions. Figure 5 shows representative adversarial examples from the Case A white-box attack.

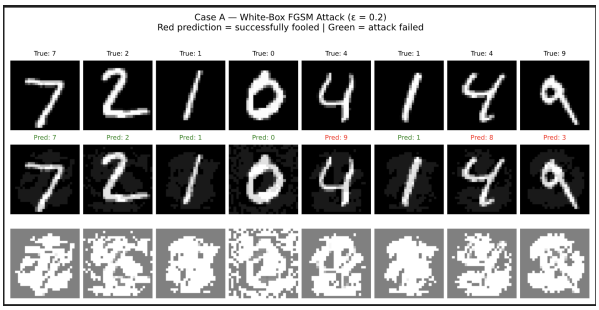

Figure 5: Adversarial examples, Case A white-box FGSM ($\varepsilon = 0.20$). Red: successful misclassification. Green: attack failed.

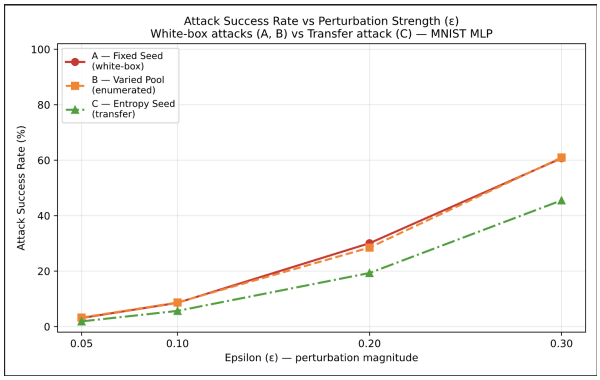

Figure 6: ASR versus perturbation budget for all three cases. White-box attacks (A, B) substantially outperform the transfer attack (C) at every ε.

Table 2: Attack Success Rate (%) by seeding condition and ε.

	Perturbation budget (ε)			
Attack paradigm	0.05	0.10	0.20	0.30
A — White-box (exact reconstruction)	3.1	8.6	30.1	60.7
B — Enumerated → white-box	3.2	8.7	28.5	61.0
C — Transfer (surrogate)	1.9	5.7	19.3	45.5
Gap (A − C)	1.2	2.9	10.8	15.2

Cases A and B are security-equivalent. Once the correct seed is recovered, directly in Case A, via five training runs in Case B, the attacker holds a perfect model replica (agreement = 1.00) and achieves ASR of $\sim 61\%$ at $\varepsilon = 0.30$ and $\sim 30\%$ at $\varepsilon = 0.20$. A small, public seed pool provides no practical security: the enumeration cost is negligible.

Case C substantially degrades attack effectiveness. The surrogate achieves only partial agreement with the victim ($\approx$0.9950), reflecting gradient misalignment between differently initialised models. Goodfellow et al. [4] explained this mechanism: adversarial perturbations transfer across models because similarly trained classifiers learn similar linear boundaries, but transfer degrades as initialisation diverges. At $\varepsilon = 0.30$, ASR falls from 60.7% to 45.5% (-15.2 percentage points). At $\varepsilon = 0.20$, the reduction is 10.8 points. This gap is consistent with the white-box/black-box accuracy difference reported by Madry et al. [5]: their MNIST model fell to 89% under white-box PGD but retained above 95% accuracy against black-box transfer attacks, a gap driven by exactly the same gradient-access mechanism.

Analyzing the absolute ASR values. MNIST with dropout is an intentionally simple setting. Sen and Dasgupta [6] observed Top-1 errors of 94–97% under FGSM at $\varepsilon = 0.10$ on ImageNet models without adversarial hardening—far higher absolute ASR than our results at the same ε (8.6%). The transfer gap (the security-relevant quantity) is expected to persist or widen on harder tasks, since more complex models exhibit greater parameter-space divergence for the same seed mismatch.

6. Discussion

6.1. A Unified View of the Three Cases

The three cases studied here are not independent. Reproducibility, as required by Pineau et al. [1], demands seed disclosure, which collapses the attack model from Case C to Case A. Accuracy, as predicted by Scardapane and Wang [2] and confirmed by Experiment 1, is unaffected by which seeding strategy is used. Security, as measured by the ASR gap in Experiment 2, is entirely determined by whether the seed is public or private.

This produces a clear and practical policy: *the algorithm is public; the seed is private.* Architectures, parameters, and training code can be disclosed without much security cost, none of these alone enables model reconstruction. The seed is the sole remaining secret. Ahmed and Lofstead [3] recognised that seed control is necessary for reproducibility, but proposed making seeds universally accessible for replay. Our results argue for private replay instead: the seed is logged for authorised auditors, not published, preserving both reproducibility and the security benefit of entropy seeding.

6.2. Entropy Seeding as a Zero-Cost Passive Defence

Existing adversarial defences impose measurable costs. Adversarial training [5] substantially prolongs training and reduces clean-input accuracy. Input transformation defences and gradient masking [7] add inference-time computation and can be bypassed by adaptive attacks. Entropy seeding imposes none of these costs: it requires replacing a fixed seed constant with a single `os.urandom` call, incurs zero additional training time, and produces no accuracy reduction.

The defence is targeted as it closes the model reconstruction vector that all other defences presuppose has already been resolved. Used alongside adversarial training or robustness certification, it addresses the access-acquisition phase that precedes white-box attacks, complementing rather than replacing existing methods.

6.3. Limitations

The experiments are conducted on MNIST with a shallow MLP. Larger architectures and harder datasets may exhibit greater seed-induced accuracy variance, potentially shifting the efficiency analysis. Only single-step FGSM is evaluated, iterative methods such as PGD [5] may produce different transfer-gap profiles. Finally, the 2^{32} seed space, while non-enumerable at training cost for this model, may be insufficient for adversaries with very large compute budgets, 64-bit entropy via `os.urandom(8)` would eliminate this concern at no additional cost.

7. Conclusion

I investigated the joint effects of three random number generation seeding strategies on neural network reproducibility, training accuracy, and adversarial security. Three conclusions are established experimentally.

Seeding strategy does not affect model quality or convergence. Mean test accuracy spans only 0.09%

across conditions and loss curves are indistinguishable by epoch 10, consistent with the seed-agnostic convergence predicted by Scardapane and Wang [2].

A known or enumerable seed eliminates deployment security. Publishing a fixed seed [1] or a small seed pool gives an attacker exact model reconstruction at negligible cost, enabling full white-box FGSM access with ASR up to 61% at $\varepsilon = 0.30$ in our experiment.

Hardware entropy seeding is a passive, zero-cost defence. By placing the seed in a 2^{32}-element space, entropy seeding forces the attacker into the transfer paradigm, reducing ASR by 15.2 percentage points at $\varepsilon = 0.30$ and 10.8 points at $\varepsilon = 0.20$, a gap consistent with the white-box/black-box performance difference established by Madry et al. [5] and the gradient-based attack hierarchy of Chen et al. [7].

I recommend treating the training seed as a deployment secret. While os.urandom enhances security by preventing predictable PRNG states, it makes public reproducibility impossible. If reproducibility is a priority, one must use a fixed seed. However, this seed must be kept private and never published in public repositories to avoid security risks.

Code Availability

`https://github.com/OvaisQazi/Research-Paper-1`

References

[1] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Lariviere, A. Beygelzimer, F. d'Alche Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research(a report from the neurips 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[2] S. Scardapane and D. Wang, "Randomness in neural networks: an overview," *WIREs Data Mining and Knowledge Discovery*, vol. 7, no. 2, p. e1200, 2017.

[3] H. Ahmed and J. Lofstead, "Managing randomness to enable reproducible machine learning," in *Proceedings of the 5th International Workshop on Practical Reproducible Evaluation of Computer Systems*, P-RECS '22, (New York, NY, USA), p. 15–20, Association for Computing Machinery, 2022.

[4] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2015.

[5] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2019.

[6] J. Sen and S. Dasgupta, "Adversarial attacks on image classification models: Fgsm and patch attacks and their impact," 2023.

[7] Y. Chen, M. Zhang, J. Li, and X. Kuang, "Adversarial attacks and defenses in image classification: A practical perspective," in *2022 7th International Conference on Image, Vision and Computing (ICIVC)*, pp. 424–430, 2022.